Complete RAG Pipeline

Documents → Retrieval → Augmentation → Generation

How External Knowledge Reaches an LLM

MODULE 2 · LECTURE 2 | DURATION · 60 MIN | SYLLABUS · 2.1 INTRODUCTION TO RAG

TOPICS COVERED

- Two Phases
- Document Loading
- Text Extraction
- Metadata
- Chunking
- Chunk Size & Overlap
- Embeddings
- Vector Database
- Semantic Retrieval
- Top-k
- Prompt Augmentation
- Generation

LECTURE MAP · FROM WHY TO HOW

WHERE ARE WE — AND A 30-SECOND RECALL

Where We Are, and What RAG Stands For

THE ROAD THROUGH MODULE 2

LECTURE 1 · WHY RAG?

LLM limitations | External knowledge | Basic architecture

↓

LECTURE 2 · HOW RAG WORKS — YOU ARE HERE

Documents | Chunking | Embeddings | Vector Database | Retrieval | Prompt Augmentation | Generation

↓

LECTURE 3 · MAKING RETRIEVAL BETTER

Important distinction: Lecture 2 = **pipeline mechanics**. Lecture 3 = **retrieval quality and optimization**.

QUICK RECALL FROM LECTURE 1

Ask the class: **"What does RAG stand for?"**

R **Retrieval**
find relevant external information

A **Augmented**
add retrieved information to the model's context

G **Generation**
generate the final response

```
RAG = Retrieve + Augment + Generate
```

Then ask: "Does normal RAG necessarily retrain the LLM?" **Answer: No.**

MAIN ARCHITECTURE SLIDE · THE MAP OF EVERYTHING THAT FOLLOWS

The Entire RAG Pipeline

This one diagram is the heart of Lecture 2.

KNOWLEDGE SIDE — INDEXING

PDFs · Web · FAQs · Policies · Database

1. Document Loader
2. Text Extraction
3. Cleaning
4. Chunking
5. Embedding Model · SHARED
6. Chunk Embedding Vectors
7. Vector Database

QUERY SIDE — QUESTION TIME

runs for every user question

1. User Question
2. Embedding Model · SHARED
3. Query Vector
4. Search Vector Database
5. Relevant Top-k Chunks
6. Construct Augmented Prompt
7. LLM
8. Grounded Final Answer

BLACKBOARD · DOCUMENT → CHUNK → EMBEDDING → VECTOR DB

BLACKBOARD · QUERY → EMBEDDING → SEARCH → TOP-K

TWO MAJOR PHASES · AN ESSENTIAL DISTINCTION INDEXING VS QUERY TIME

RAG Has Two Major Phases

PHASE 1

Indexing / Knowledge Preparation

Usually performed when documents are added or updated.

`Documents` → `Extract` → `Clean` → `Chunk` → `Embed` → `Store`

PHASE 2

Retrieval + Generation

Performed when a user asks a question.

`Question` → `Embed` → `Retrieve` → `Augment` → `Generate`

Why separate the two phases?

Suppose you have `10,000 documents`. Would you recompute embeddings for all 10,000 documents **every time someone asks a question? No.**

`Documents` → `Prepare once` → `Store vectors`

Then every query can search those stored vectors. If documents change, the **affected index entries** can be updated.

Key idea: document embeddings are generally prepared **ahead of time** and **reused**.

MEMORY ANCHOR — SAY IT ALOUD

"Index first.
Retrieve later."

Blackboard: **Two Phases — Indexing / Query Time.**

PART A — DOCUMENT INGESTION · STEPS 1-2 KNOWLEDGE SOURCES & THE DOCUMENT LOADER

Where Knowledge Enters the System

Step 1 — A RAG system may use:

- PDF Files
- Word Documents
- Web Pages
- HTML
- Text Files
- FAQs
- Product Manuals
- Research Papers
- Policy Documents
- Database Records
- Support Documentation

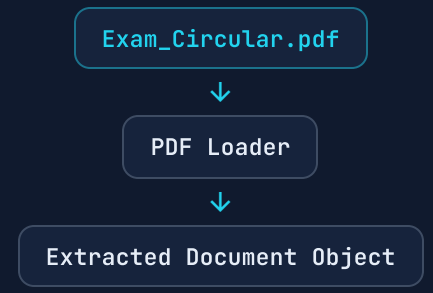
RUNNING UNIVERSITY EXAMPLE

- Academic_Regulations.pdf
- Exam_Circular_Oct_2026.pdf
- Fee_Rules.pdf
- Student_Handbook.pdf

These form our external knowledge source.

Step 2 — The Document Loader

A document loader reads information from the source.



The loader may preserve:

Text + Page Number + Filename + Title + Other Metadata

Important: a document loader does **not "understand"** the document. Its first job is simply to extract usable content and metadata.

PART A — DOCUMENT INGESTION · EXTRACTION & CLEANING

FROM RAW FILE TO USABLE TEXT

Text Extraction and Cleaning

STEP 3 Text Extraction

An original PDF may contain:

Text | Tables | Images | Headers | Footers | Page Numbers | Formatting

The RAG pipeline often needs: **machine-readable text.**

PDF PAGE → EXTRACTED TEXT

```
"The last date for Semester VIII examination-form submission is 18
October 2026."
```

Caveat: scanned-image PDFs may require a separate **OCR / document-understanding** step — we will not go deeply into OCR here.

STEP 4 Cleaning the Text

Raw document text may contain:

Repeated headers | Repeated footers | Page numbers | Broken whitespace | Formatting artifacts | Navigation text | Duplicate text

CLEANING MAY TRANSFORM

```
--- UNIVERSITY EXAMINATION CELL ---
Page 7 of 20
Students must submit...
```

↓

```
Students must submit...
```

Key point: bad document preprocessing creates **bad material for later retrieval.**

PART A — DOCUMENT INGESTION · METADATA

DO NOT STORE ONLY TEXT

Every Chunk Carries Metadata

A chunk may also carry:

source	= Exam_Circular_Oct_2026.pdf
page	= 4
department	= Examination
year	= 2026
document_type	= Circular

EXCELLENT MEMORY LINE

”The vector tells us what the text means. Metadata tells us where it came from.”

Why is metadata useful?

- **Identifying the source** — which file, which page
- **Citation** — answers can point back to evidence
- **Filtering** — restrict search by type, year, department
- **Access control** — who may see which documents
- **Debugging** — trace why a chunk was retrieved

Tell students: metadata answers **"where did this come from?"** — the vector answers **"what does it mean?"**

PART B — CHUNKING · WHY & WHAT

THE RETRIEVAL UNIT OF RAG

Why Do We Need Chunking — and What Is a Chunk?

THE PROBLEM

Would we embed all 100 pages as one vector?

Usually **not ideal** — one enormous vector representation would **mix many unrelated topics**.

EXAMPLE PDF — TOPIC MAP

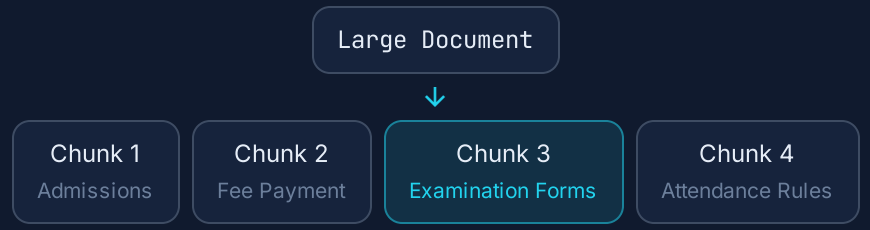
Page 1-10	Admission
Page 11-25	Fees
Page 26-40	**Examinations**
Page 41-60	Attendance
Page 61-100	Other Regulations

Question: "What is the examination deadline?" — we want the relevant **examination section**, not the entire document.

THE DEFINITION

What Is a Chunk?

A chunk is a **smaller segment of a larger document** used as an **individual retrieval unit**.



At retrieval time: we retrieve **chunks**, not necessarily entire documents.

PART B — CHUNKING · WORKED TEXT ONE PARAGRAPH, THREE CHUNKS

A Simple Chunking Example

ORIGINAL TEXT — EXAMINATION INSTRUCTIONS

Students must complete examination registration through the student portal. **The last date is 18 October 2026.** Late forms require approval from the Examination Cell. Hall tickets will be released after eligibility verification.

↓

1 Chunk 1 — Registration

"Students must complete examination registration through the **student portal**."

2 Chunk 2 — Deadline & Late Forms

"The last date is **18 October 2026**. Late forms require approval from the **Examination Cell**."

3 Chunk 3 — Hall Tickets

"Hall tickets will be released after **eligibility verification**."

Now a deadline question — *"What is the last date for examination registration?"* — can retrieve **Chunk 2** and nothing else.

PART B — CHUNKING · SIZE A DESIGN CHOICE, NOT A UNIVERSAL CONSTANT

Chunk Size — Small vs Large

What is chunk size?

Chunk size = how much text goes into **one chunk**.

SMALL CHUNK

≈ a short paragraph

LARGE CHUNK

≈ several paragraphs

Say it to students: "Chunk size is a **design choice**, not a universal constant." Don't give one magic number.

The simplified trade-off

Smaller chunks

Benefit: more **focused** information

Problem: context may be **incomplete**

Larger chunks

Benefit: more **surrounding context**

Problem: more **unrelated information**

```
Too small → context may break
Too large → retrieval may become less focused
```

Advanced chunk tuning comes in Lecture 3 — today we only build the concept.

PART B — CHUNKING · OVERLAP WHEN A SENTENCE CROSSES THE BOUNDARY

What Is Chunk Overlap — and Why Use It?

WITHOUT OVERLAP meaning can get split awkwardly

CHUNK 1

"Students must submit examination forms **before...**"

CHUNK 2

"**...18 October 2026.** Late forms require approval..."

The deadline sentence is **cut in half** — neither chunk alone carries the full rule.

WITH OVERLAP some information appears in neighboring chunks

CHUNK 1

"Students must submit examination forms **before 18 October 2026.**"

CHUNK 2 · starts with the repeated tail

"**before 18 October 2026.** Late forms require approval..."

Overlap can help preserve: **sentence continuity**, **context across boundaries**, and **important information near chunk edges**.

More Overlap → More Chunks → More Embeddings → More Storage → Potential Duplication

Overlap is useful, but **more overlap is not automatically better** — tuning belongs to Lecture 3.

PART C — EMBEDDINGS · MOTIVATION

FROM CHUNKS TO MEANING

We Now Have Chunks. How Do We Search by Meaning?

TEXT A

”The online fee transaction was unsuccessful.”

TEXT B

”My payment failed.”

They use **different words**

✕ **Keyword matching** may miss some semantic relation — the two texts mean nearly the same thing, yet share almost no words.

→ We need a **numerical representation of meaning**.

The answer: Embeddings

PART C — EMBEDDINGS · DEFINITION MEANING AS GEOMETRY

What Is an Embedding — and the Embedding Model

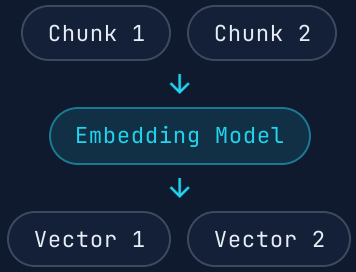
An embedding is a **numerical vector representation of text** designed so that **semantic relationships** can be reflected **geometrically** in the vector space.

`"payment failed"`	→	Embedding Model	→	`[0.21, -0.48, 0.73, ..., 0.11]`
`"transaction unsuccessful"`	→	Embedding Model	→	`[0.19, -0.44, 0.76, ..., 0.09]`

Conceptually: related texts may have **nearby or similar** vector representations — the two vectors above are close to each other.

The embedding model

Converts text into a **fixed-dimensional vector**:



A 768-DIMENSIONAL EMBEDDING

`[0.21, -0.48, 0.73, ..., 0.11]`

It contains **768** numerical components.

Do not try to interpret individual components as human-readable concepts.

PART C — EMBEDDINGS · CLARIFICATION

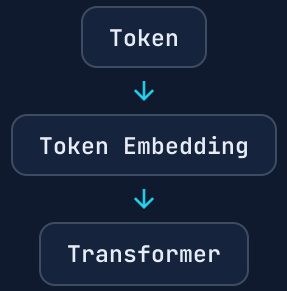
TWO DIFFERENT JOBS FOR THE WORD "EMBEDDING"

Token Embeddings vs RAG Embeddings

INSIDE THE LLM

LLM Token Embedding

Represents an **individual token** for processing inside the LLM.

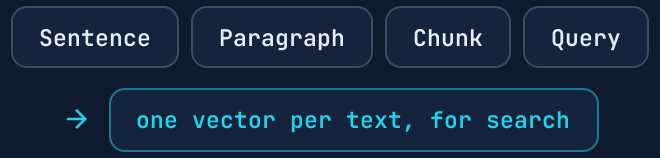


You met this in Lecture 1 — it is the LLM's internal working representation.

USED FOR RETRIEVAL

RAG / Retrieval Embedding

Represents a whole **sentence, paragraph, chunk or query** as a vector used for retrieval.



This is what today's pipeline stores in the vector database.

Key distinction: both are vectors, but they **serve different purposes** — and neither is the same thing as a token ID.

PART C — EMBEDDINGS · THE GOLDEN RULE

ONE SPACE FOR DOCUMENTS AND QUERIES

Documents and Queries Must Be Embedding-Compatible



Important: the query and document vectors must live in a **compatible embedding space**. Using arbitrary unrelated embedding models for indexing and querying would make similarity **meaningless**.

SEMANTIC SEARCH INTUITION

KNOWLEDGE CONTAINS

"Tuition transactions that have been debited but not reflected require payment reconciliation."

USER ASKS

"My fee was deducted but portal still shows unpaid."

Words are **not identical** — but semantic meaning **overlaps**, so `Query Vector ≈ Document Chunk Vector`. Therefore retrieval can find it.

MEMORY ANCHOR — SIMPLIFIED BUT EXCELLENT

Keyword search asks: *"Do the words match?"*

Semantic retrieval asks: *"Do the meanings match?"*

PART D — VECTOR DATABASE THE INDEX THAT MAKES SEARCH FAST

What We Store — and What a Vector Database Is

For every chunk, a RAG index may store:

Chunk Text + Embedding Vector + Metadata

TEXT	"The last date is 18 October 2026."
VECTOR	[0.31, -0.55, ..., 0.17]
METADATA	source = Exam_Circular.pdf page = 4 · year = 2026

Ask the class: **what is stored with a chunk besides its vector?** — text and metadata, never the vector alone.

What is a vector database?

A vector database / index **stores vector representations** and supports **efficient retrieval of vectors similar** to a query vector.

EXAMPLES STUDENTS MAY ENCOUNTER

FAISS · Chroma · Milvus · Weaviate · Pinecone

But emphasize: RAG is **not equal** to any particular vector-database product — these are **implementations** of one concept.

PART D — VECTOR DATABASE · COMPARISON

TWO DIFFERENT WAYS TO LOOK THINGS UP

Traditional Lookup vs Vector Retrieval

TRADITIONAL LOOKUP		VECTOR RETRIEVAL
exact field / value	⇄	**semantic similarity**
`student_id = 25`	⇄	"Which text means something **similar**?"
structured data	⇄	often **unstructured** text representations
deterministic filtering	⇄	**similarity ranking**

But clarify: modern RAG systems often **combine metadata filtering with vector retrieval**. Advanced hybrid retrieval comes later, if needed.

PART E — QUERY-TIME RETRIEVAL FROM QUESTION TO RANKED MATCHES

A Question Arrives — Similarity Search Begins

A user asks:

"What is the last date for examination registration?"

Question

↓

Embedding Model

↓

Query Vector

```
q = [0.24, -0.41, ..., 0.61]
```

This query vector can be **compared against stored chunk vectors**.

Compare with stored vectors — find the most similar chunks

HIGH **Chunk A** — Exam registration deadline

LOW **Chunk B** — Fee refund

VERY LOW **Chunk C** — Library opening hours

Do not teach cosine-similarity mathematics deeply here. "Lecture 3 will explain how similarity and retrieval quality are measured."

PART E — QUERY-TIME RETRIEVAL · TOP-K HOW MUCH EVIDENCE DO WE TAKE?

Top-k Retrieval — and What Comes Back

`k = 3` **Then retrieve the three best-ranked chunks**

1. Examination deadline chunk
2. Late-form procedure chunk
3. Hall-ticket eligibility chunk

Top-k means return the **k most relevant retrieved items** according to the retrieval system. Do **not** say higher k is always better — that trade-off belongs to Lecture 3.

What comes back from retrieval?

The retriever can return chunks **plus** source, page, similarity information, metadata:

CHUNK
"The last date is 18 October 2026."

SOURCE
Exam_Circular_Oct_2026.pdf

PAGE
4

This evidence will now be **supplied to the LLM**.

What Is Prompt Augmentation?

ORIGINAL PROMPT

What is the examination-registration deadline?

↓

AFTER RETRIEVAL — THE AUGMENTED PROMPT

CONTEXT:

[Chunk 1]
The last date for Semester VIII examination registration is **18 October 2026**.

[Chunk 2]
Late forms require approval from the Examination Cell.

QUESTION:

What is the examination-registration deadline?

This — **context + question together** — is an **augmented prompt**, and it is what gets sent to the LLM.

CONTEXT ALONE IS NOT ENOUGH

A Better RAG Prompt — Instructions With Context

SYSTEM:
You are a university helpdesk assistant.

INSTRUCTIONS:
Answer the question using the supplied context. If the answer is not supported by the context, state that the information is unavailable.

CONTEXT:

```
[Retrieved Chunk 1]
[Retrieved Chunk 2]
[Retrieved Chunk 3]
```

QUESTION:

```
What is the deadline for Semester VIII examination registration?
```

Augmented Prompt → LLM

Why give instructions with context?

Simply placing retrieved text beside a question may not always be enough. Instructions can tell the LLM to:

- **use supplied evidence** as the basis of the answer
- **avoid unsupported claims** beyond the context
- **answer concisely**
- **provide source references** if required
- **admit** when context is insufficient

Important: retrieval **supplies information**. Prompt design tells the generator **how to use it**.

PART G — GENERATION THE LLM WRITES, THE WEIGHTS STAY STILL

The Generator LLM — and Retrieval Does Not Write Into the Weights

Now the LLM receives:

`User Question` + `Retrieved Context` + `Instructions`

Then uses its language capability to:

- understand the question
- synthesize relevant evidence
- organize the answer
- generate natural language

EXAMPLE ANSWER

"According to the supplied examination circular, the last date for Semester VIII examination registration is **18 October 2026**."

Important recall: context, not parameters

`Retrieved Context` → `Input Context` → `LLM`

The retrieved document normally does **not** become a permanent model parameter during ordinary RAG inference. The next user may retrieve a **completely different context**:

`Query A`	→	Exam Circular
`Query B`	→	Fee Policy
`Query C`	→	Attendance Rule

Same LLM. Different context.

WORKED EXAMPLE · STEP BY STEP ONE QUESTION, END TO END

Complete Worked Example

KNOWLEDGE BASE

- Exam_Circular.pdf
- Fee_Policy.pdf
- Attendance_Rules.pdf

USER ASKS

"What happens if my examination form is submitted after 18 October?"

1. **The question is embedded**
   the query vector lives in the same space as the chunk vectors
2. **The vector index retrieves:**
   "The last date is 18 October. Late forms require approval from the Examination Cell."
3. **The retrieved chunk is added to the prompt**
   context above, question below — the augmented prompt
4. **The LLM generates:**
   "Forms submitted after 18 October require approval from the Examination Cell, **according to the retrieved examination circular**."

That is the complete RAG process — every step from question to grounded answer in one pass.

MAJOR RECAP · OFFLINE + ONLINE

THE WHOLE LECTURE IN ONE PICTURE

One Query Through the Entire System

OFFLINE — DONE ONCE PER DOCUMENT SET

Exam_Circular.pdf

↓

Load

↓

Extract Text

↓

Clean

↓

Chunk

↓

Embed

↓

Store in Vector DB

ONLINE — EVERY TIME A QUESTION ARRIVES

"What is the deadline?"

↓

Embed Question

↓

Search Vector DB

↓

Retrieve Relevant Chunk

↓

Build Augmented Prompt

↓

LLM

↓

"18 October 2026"

Tell students: *"If you can explain this diagram, you understand the basic RAG pipeline."*

FAILURE MAP & LECTURE BOUNDARIES HONESTY ABOUT WHAT CAN GO WRONG

Where Can the Pipeline Fail — and What We Did Not Cover

Failure can start long before generation

Bad Document → Bad extracted text → Bad chunk → Poor embedding / index → Wrong retrieval → Wrong context → Poor answer

Or — even with correct retrieval:

Correct retrieval → LLM misuses context → Poor answer

Important distinction: RAG failures can occur in **retrieval** or **generation** — **or before either of them**. Lecture 3 will teach how to diagnose these systematically.

What we deliberately did NOT cover today

Naming the boundaries keeps the lecture honest:

- Which embedding model is best?
- What should chunk size be?
- How much overlap?
- Cosine similarity mathematics?
- Dense vs sparse retrieval?
- Hybrid retrieval?
- How many chunks to retrieve?
- Reranking?
- Recall@k?
- Faithfulness?
- Hallucination evaluation?

Then say: *"Those are retrieval-quality questions — **and that is Lecture 3.**"*

CHECK YOUR UNDERSTANDING CLASSROOM ACTIVITY + TRUE/FALSE

Activity: Trace the Pipeline — and Bust the Myths

GIVE THIS DOCUMENT

University Examination Rules

1. Examination forms close on **18 October**.
2. Hall tickets are available five days before examination.
3. Students below the attendance threshold must seek approval.
4. Late examination forms require Examination Cell approval.

Question: *"Can I submit the examination form after the deadline?"*

DOCUMENT	The Examination Rules.
LIKELY USEFUL CHUNK	Rule 1 + Rule 4.
WHAT IS EMBEDDED?	The chunks and the query.
WHAT IS RETRIEVED?	Relevant rules.
WHAT IS AUGMENTED?	The prompt.
WHO GENERATES?	The LLM.

COMMON MISCONCEPTIONS — RUN AS TRUE/FALSE

1	The whole PDF must be inserted into the prompt for every question.	FALSE
2	Chunking divides documents into smaller retrieval units.	TRUE
3	Embeddings are human-readable summaries.	FALSE
4	A vector database stores only vectors — never text or metadata.	FALSE
5	User queries can also be embedded.	TRUE
6	Top-k means retrieve the k highest-ranked results.	TRUE
7	Retrieval permanently teaches the LLM the document.	FALSE
8	RAG quality depends only on the LLM.	FALSE

Answer key: F · T · F · F · T · T · F · F

MEMORY DEVICE & VOCABULARY

THE STRONGEST MEMORY DEVICE FOR LECTURE 2

The RAG Pipeline in Seven Words

Load	Chunk	Embed	Store	Retrieve	Augment	Generate
documents enter the system	break into useful pieces	convert text into vectors	save vectors + text + metadata	find relevant pieces	add them to the prompt	LLM produces the response

Have students repeat it — then map every term

Document
original knowledge source

Chunk
smaller piece of the document

Embedding
numerical vector representation

Embedding Model
converts text to vectors

Vector DB / Index
stores & searches vectorized chunks

Query Embedding
vector of the user's question

Retriever
finds relevant chunks

Top-k
number of highest-ranked items returned

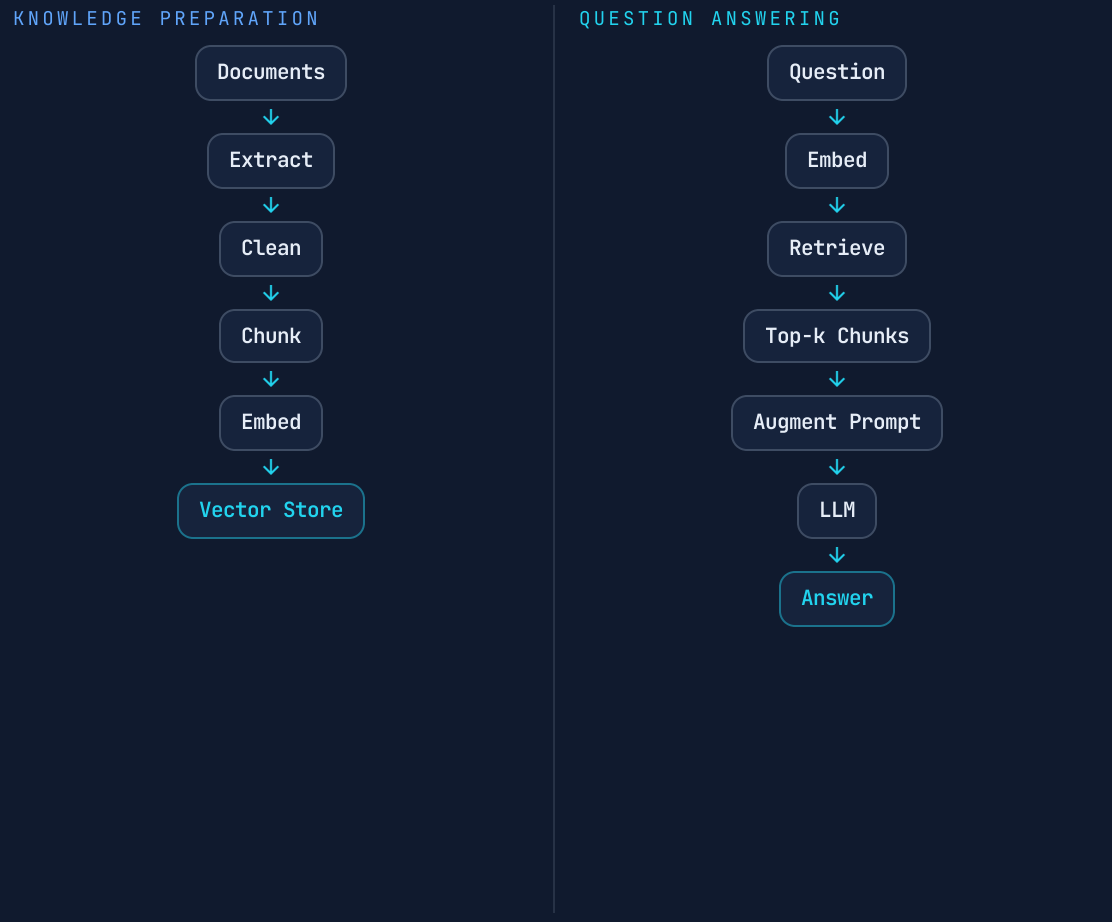
Context
retrieved evidence supplied to the LLM

Generator
LLM producing the final answer

`Load → Chunk → Embed → Store → Retrieve → Augment → Generate`

SUMMARY & BRIDGE WHAT WE BUILT — AND THE NEXT ENGINEERING PROBLEM

Final Summary — and Bridge to Lecture 3



ONE-LINE SUMMARY

A RAG system **prepares documents as searchable semantic representations**, **retrieves evidence relevant to a query**, and **supplies that evidence to an LLM** for generation.

Create the next engineering problem. Ask:

"What if the vector database retrieves the wrong chunk?"

Correct Question → Wrong Retrieval → Wrong Context → Potentially Wrong Answer

NEXT — LECTURE 3 · RETRIEVAL QUALITY

- Similarity Search
- Top-k
- Reranking
- RAG Evaluation
- Chunking Strategies
- Metadata Filtering
- Hallucination Reduction

Closing: *"Today we built the RAG engine. Next lecture, we learn how to make its retrieval reliable."*